

```
1 using CustomProjectRPG.Core.Utilities;
2 using CustomProjectRPG.Interfaces;
3 using System;
4 using System.Collections.Generic;
5 using System.Linq;
6 using System.Text;
7 using System.Text.Json;
8 using System.Text.Json.Serialization;
9 using System.Threading.Tasks;
10 using static CustomProjectRPG.Entities.Enum.Enum;
11
12 namespace CustomProjectRPG.Dialogue
13 {
14
15     public class DialogueManager : ISerializeJSON<DialogueManager>
16     {
17         private Dictionary<string, Dictionary<string, string>>
18             _npcDialogues; // NPC ID -> Dialogue ID -> Text
19         private ICorruption _corruption;
20         private string _current_npcId;
21         private string _currentDialogueId;
22
23         public event Action<string> DialogueChanged; // Observer pattern
24         // for UI updates
25
26         /// Initializes a new DialogueManager with a corruption reference
27         /// and loads dialogues from JSON.
28
29         public DialogueManager(ICorruption corruption, string
30             jsonDialogues = null)
31         {
32             _corruption = corruption ?? throw new ArgumentNullException
33                 (nameof(corruption));
34             _npcDialogues = new Dictionary<string, Dictionary<string,
35                 string>>();
36             _current_npcId = string.Empty;
37             _currentDialogueId = string.Empty;
38
39             if (!string.IsNullOrEmpty(jsonDialogues))
40             {
41                 LoadDialoguesFromJson(jsonDialogues);
42             }
43
44             /// Loads dialogue data from a JSON string, organized by NPC ID.
45
46             private void LoadDialoguesFromJson(string json)
```

```
44     {
45         var dialogues = JsonSerializer.Deserialize<Dictionary<string,
46             Dictionary<string, string>>>(json)
47             ?? throw new JsonException("Invalid JSON format for
48             dialogues.");
49         _npcDialogues = dialogues;
50     }
51     /// Retrieves available dialogue options for the current NPC
52     based on corruption state.
53     public List<string> GetDialogueOptions()
54     {
55         if (!_npcDialogues.ContainsKey(_current_npcId) || !
56             _npcDialogues[_current_npcId].Any())
57         {
58             return new List<string> { "No dialogue available for this
59             NPC." };
60         }
61         var options = new List<string>();
62         var npcDialogues = _npcDialogues[_current_npcId];
63         if (npcDialogues.TryGetValue("normal", out var
64             normalDialogue))
65         {
66             options.Add(normalDialogue);
67         }
68         if (_corruption.IsDialogueCorrupted() &&
69             npcDialogues.TryGetValue("corrupted", out var
70             corruptedDialogue))
71         {
72             options.Add(corruptedDialogue);
73         }
74         if (_corruption.IsNoResponse() && npcDialogues.TryGetValue
75             ("noResponse", out var noResponseDialogue))
76         {
77             options.Clear();
78             options.Add(noResponseDialogue);
79         }
80         return options.Count > 0 ? options : new List<string> { "No
81             valid dialogue options." };
82     }
83     /// Sets and displays the current dialogue for the specified NPC,
84     triggering an event.
```

```
82
83     public void ShowDialogue(string npcId, string dialogueId)
84     {
85         if (!_npcDialogues.ContainsKey(npcId) || !_npcDialogues      ↗
            [npcId].ContainsKey(dialogueId))
86         {
87             throw new ArgumentException($"Invalid NPC ID or dialogue      ↗
                ID: {npcId}/{dialogueId}.", nameof(dialogueId));
88         }
89         _current_npcId = npcId;
90         _currentDialogueId = dialogueId;
91         OnDialogueChanged();
92     }
93
94
95     /// Gets the current dialogue text for the active NPC.
96
97     public string GetCurrentDialogue()
98     {
99         if (string.IsNullOrEmpty(_current_npcId) ||                  ↗
            string.IsNullOrEmpty(_currentDialogueId) ||
100            !_npcDialogues.ContainsKey(_current_npcId) || !          ↗
            _npcDialogues[_current_npcId].ContainsKey                ↗
            (_currentDialogueId))
101         {
102             return "No dialogue selected.";
103         }
104         return _npcDialogues[_current_npcId][_currentDialogueId];
105     }
106
107
108     /// Serializes the current dialogue state to JSON.
109
110     public string Serialize()
111     {
112         var data = new DialogueData
113         {
114             Current_npcId = _current_npcId,
115             CurrentDialogueId = _currentDialogueId
116         };
117         return JsonSerializer.Serialize(data);
118     }
119
120
121     /// Deserializes the dialogue state from JSON.
122
123     public void Deserialize(string json)
124     {
125         var data = JsonSerializer.Deserialize<DialogueData>(json)
```

```
...\\Admin\\CustomProjectRPG\\Dialogue\\DialogueComponent.cs 4
126      ?? throw new JsonException("Invalid JSON format for  ↗
      dialogue state.");
127      if (_npcDialogues.ContainsKey(data.Current_npcId) &&  ↗
      _npcDialogues[data.Current_npcId].ContainsKey  ↗
      (data.Current_dialogueId))
128      {
129          _current_npcId = data.Current_npcId;
130          _current_dialogueId = data.Current_dialogueId;
131      }
132  }
133
134  private void OnDialogueChanged()
135  {
136      DialogueChanged?.Invoke(GetCurrent_dialogue());
137  }
138
139  private class DialogueData
140  {
141      [JsonPropertyName("current_npcId")]
142      public string Current_npcId { get; set; }
143
144      [JsonPropertyName("current_dialogueId")]
145      public string Current_dialogueId { get; set; }
146  }
147  }
148 }
149
```